

Executive Summary

Prompt design for Google's Gemini models (3.x series, including 3.1+) centers on **clarity, structure, and grounding**. Use explicit, concise instructions and structured formats (e.g. XML/Markdown tags) to distinguish tasks, context, and roles [17†L863-L872] [28†L21-L29]. Include **few-shot examples** to demonstrate desired output formats and reduce ambiguity [14†L503-L511]. By default Gemini 3 models expect *short, direct responses* – to change verbosity or style, explicitly request it in the prompt [17†L869-L878] [28†L27-L34]. Place critical directives (personas, constraints, output format) at the top, and put your main question last, especially when providing long context [6†L1414-L1422] [17†L879-L887]. Leave most generation parameters at their defaults (temperature=1.0, top_k/p unchanged) unless you have a specific reason, as Gemini 3's reasoning was tuned for those defaults [6†L1346-L1354] [35†L13-L22]. Leverage Gemini's massive context window (up to ~1M tokens [30†L248-L257]) by feeding all relevant information up front and anchoring your question with phrases like *"Based on the information above..."* [6†L1478-L1487] [17†L878-L887]. For complex or multi-step tasks, instruct the model to plan or verify steps (Gemini can auto-generate internal "thinking" but explicit scaffolding can help) [17†L919-L924] [28†L48-L57]. Always test and iterate: if outputs are misaligned, refine instructions (favor positive directives over vague negatives [6†L1378-L1386] [19†L141-L149]), adjust examples, or simplify the task. The tables and examples below compare effective vs. ineffective prompts, highlight parameter guidelines, and summarize recommended templates.

[17†L863-L872] [14†L503-L511] *Figure: Example structured prompt and few-shot guidance for Gemini.*

Prompt Structure and Roles

- **System (Persona) vs. User:** In Gemini's chat or agent API, set a **system instruction** or opening block to define the assistant's role, tone, and constraints [17†L969-L977] [28†L79-L87]. For example, start with a section like `<role>You are a helpful scientific assistant.</role>` or a heading `# Identity` to anchor the persona [17†L969-L977] [28†L80-L88]. Put any hard constraints (e.g. "Cite sources", "Don't use slang") here as well to ensure the model keeps them in mind.
- **Structured Formatting:** Use clear delimiters (XML-style tags or Markdown headings) to partition the prompt into context, instructions, and questions [17†L863-L872] [28†L79-L87]. For example:

```
<context>
[Insert relevant text or data here]
</context>
<task>
[Ask the model's specific question here]
</task>
```

This avoids confusion over what parts are “instructions” vs. “data”. Consistency is critical – once you choose a format (e.g. XML tags or `# Headings`), use it uniformly throughout the prompt [【17†L863-L872】](#) [【28†L79-L88】](#) .

- **Place Instructions Early:** Put the *goal and key instructions first*, then any contextual data or examples, and ask the specific question at the end. Gemini models respond best when the **main task is clearly stated up front**, rather than buried in long text [【19†L39-L47】](#) [【17†L878-L887】](#) . (See *Example 1* vs *2* below.)
- **Anchoring Context:** If you feed a large block of context (document, conversation history, etc.), always “bridge” to your question. Phrases like “Based on the information above...” help the model lock onto that context [【6†L1478-L1487】](#) [【17†L878-L887】](#) .

Example (Poor vs. Good System Role):

-  “You are a friendly helper. Explain in detail why X happened.” (System role is generic; may slip into irrelevant verbosity.)
-  “# Identity: You are an expert historian known for clear, concise summaries.” (Defines persona and style.)

Example (Structure):

```
<role>You are a customer-support agent for a tech company.</role>
<instructions>
1. Provide step-by-step troubleshooting.
2. Be polite and professional.
</instructions>
<context>
User reported: The new app crashes when opened after the latest update.
Relevant facts: The user is on Android 12. The error message reads "App has stopped".
</context>
<task>
Give the user a clear action plan to fix the crash.
</task>
<final_instruction>
Answer concisely; use bullet points for steps.
</final_instruction>
```

Above: System role & instructions define persona and constraints at top, context in middle, then explicit task and final formatting guide.

Few-Shot Examples

- **Always include examples when possible.** Few-shot prompts (giving 1–3 input/output examples) signal the required format and reasoning style to the model [【14†L503-L511】](#) . Google advises that

“prompts without few-shot examples are likely to be less effective” [14†L503-L511] . For tasks needing a specific answer format (JSON, bullet list, classification, etc.), demonstrate it.

- **Keep formatting consistent.** All examples should use the *same structure and style* as you expect in the answer [14†L587-L593] . Gemini learns formatting from examples, so uniform tags and delimiters prevent misformatting.
- **Vary content thoughtfully:** Use examples that cover the range of cases or lengths you expect. Don't overload with dozens – start with a few and add if needed [14†L579-L588] . Too many examples can cause the model to “overfit” to the examples’ content [14†L579-L588] .
- **Show contrast if needed:** If you want concise answers, examples should reflect brevity (see [14] where adding succinct examples changed the answer selection).

Few-shot Example (Scope Control):

Question: Summarize the causes of World War II.

Answer (concise):

- Treaty of Versailles penalties on Germany
- Expansionist policies of Axis powers
- Failures of the League of Nations

Counterexample (zero-shot vs. few-shot):

-  Q: “Classify sentiment of this review...” (no examples given). The model might vary in output style.
-  “Classify sentiment (Positive/Negative): Review1: 'Loved it!'. Label: Positive. Review2: 'Terrible service.'. Label: Negative. Review3: [New review] Label:” – Here examples guide the format.

Chain-of-Thought and Reasoning

- **Internal vs. external reasoning:** Gemini 3.x models auto-generate an *internal chain-of-thought* when solving problems [17†L919-L924] . You typically **do not need to append “Think step-by-step”** or similar unless the task is unusually hard – the model will internally iterate. Adding “explain your reasoning” often just consumes tokens without improving quality. However, you can instruct Gemini to outline steps if you want explicit traces or if using an agentic workflow.
- **Plan and critique (advanced):** For complex tasks or agent-based uses, some developers have shown benefit in explicitly prompting the model to outline a plan before solving [28†L48-L57] . For example: “1. Break the task into subgoals. 2. Verify subgoals. 3. Solve each part.” followed by executing the plan. Gemini can follow such multi-step instructions and even “critique its own output” if prompted [28†L69-L77] , but this is optional.
- **Avoid unsupported actions:** If Gemini might hallucinate capabilities (e.g. “Retrieve current weather”), use a **split-step prompt**: first ask if the info is accessible, then proceed or halt [6†L1398-L1407] .

Prompt Example (step-by-step planning):

You are a math tutor.
 First, outline the steps to solve this. Then solve it.
 Question: If $3x+5=20$, what is x ?

Temperature and Sampling Parameters

- **Keep defaults on Gemini 3.x.** Google explicitly “*strongly recommend[s] keeping [temperature] at its default 1.0*” [6†L1346-L1354] . The model’s reasoning is tuned for that setting. Similarly, **do not lower temperature** just for factual tasks; using temperature <1 can cause looping or erratic behavior [6†L1346-L1354] .
- **Top-k / Top-p:** The API supports parameters like `topK` or `topP` , but for Gemini 3.x the guidance is to leave them at default (essentially greedy or broad sampling) as well [35†L13-L22] . Deviating can make the model unexpectedly narrow its output. Only tweak parameters if you are running controlled experiments.
- **Pragmatic tuning:** In rare cases where very short, deterministic answers are needed (e.g. extracting a fixed field), you *could* set temperature $\rightarrow 0$ (greedy) and lower `max_output_tokens` . But start with defaults, and remember this trade-off is about creativity vs. determinism, not truthfulness.
- **Output length (`max_tokens`):** Set this high enough to cover expected answer length. For very long answers (e.g. summarizing a long document), ensure `max_output_tokens` can accommodate. If the model cuts off, increase this limit (Gemini will stop once it runs out of tokens).

Table: Gemini 3.x Parameter Guidance

Parameter	Recommended Setting (Gemini 3.x)	Notes/Effects
Temperature	1.0 (default) [6†L1346-L1354]	Lower $\rightarrow$ more deterministic, but may loop or degrade.
TopK	Default (no change) [35†L13-L22]	Lower $\rightarrow$ restricts choices; higher $\rightarrow$ more variety.
TopP	1.0 (default) [35†L13-L22]	Set <1 to reduce token pool, rarely needed.
Max tokens	As needed (e.g. 512–4096+)	Must exceed expected answer length; Gemini allows very large contexts [30†L248-L257] .

Token/Length Management and Context Windows

- **Massive context window:** Gemini 3 models can handle *up to ~1 million tokens* of input [30†L248-L257] (much larger than typical LLMs). In practice, this means you can feed entire books, codebases, or long chats without truncation. Use this by including *all relevant information* up front rather than summarizing externally [30†L248-L257] [6†L1478-L1487] .

- **Order of information:** When giving lots of data, present it *first*, then your specific question *last* 【17†L878-L887】 【6†L1478-L1487】 . This ensures the model reads the context before deciding on the task.
- **Context anchoring:** After a lengthy context, prepend your question with an anchor, e.g. “*Based on the document above,*” or “*From the data provided below,*” to signal the model to reference that block 【6†L1478-L1487】 【17†L878-L887】 .
- **Avoid unnecessary content:** Even though Gemini can take very long inputs, extraneous text can still distract it. Trim irrelevant details or prior turns that aren’t needed. For dialogue, you can summarize or omit earlier pleasantries if context is too long.
- **Monolithic vs. retrieval:** Because of the huge window, many techniques like RAG or splitting into pieces become optional – you can directly ask the question over all context. But if context reaches hundreds of thousands of tokens, you may still want to focus the model (e.g. “search for topic X in this text then answer”).
- **Context in multimodal inputs:** Gemini natively accepts text, images, video, or audio. If using multimodal context, mention each modality. For example: “*The first image below shows [description]. The following transcript is [text]...*” 【17†L873-L877】 .

Safety and Alignment Prompts

- **Positive framing over “don’ts”:** When you need to restrict behavior, instruct what *to do* rather than simply forbidding actions. Google’s OpenAI guide notes “Instead of just saying what *not* to do, say what to do instead” 【19†L141-L149】 . E.g. ❌ “*Do not include any personal data.*” vs ✅ “*If personal data is requested, direct the user to our privacy policy without asking them.*” 【19†L141-L149】 .
- **Placement of constraints:** Put all **formatting limits, prohibited actions, and role definitions at the end** of your instructions block 【6†L1414-L1422】 . Gemini has a tendency to drop late constraints if they come too early, so finalize your prompt with any negative or strict rules 【6†L1414-L1422】 .
- **Grounding instructions:** To ensure the model uses provided context rather than its own assumptions, explicitly ground it: “*You must rely only on the facts given and not any outside knowledge.*” 【17†L900-L914】 【6†L1453-L1461】 . This is crucial for hypothetical or fictional scenarios.
- **Safety filters:** Gemini Enterprise has built-in content filters and system safeties (e.g. abuse monitoring 【3†L333-L342】). Familiarize yourself with [System Instructions for Safety](#) if you need domain-specific filtering or compliance. In prompt design, always avoid ambiguous wording that could trigger unwanted advice.
- **Multimodal safeguards:** If using images or audio, ensure any instructions for safe content (e.g. “No copyrighted material”) are included. Gemini Image/Veo docs have additional guidelines for content moderation.

Multi-Turn Dialogue and Context Management

- **Maintain history explicitly:** In chat or agent use, pass previous user and model messages as part of the `history` (each with roles “user”/“model”/“system”) 【40†L1398-L1407】 . This preserves continuity. For very long chats, you still benefit from the large context window, so you need not summarize aggressively early.

- **System persona persistence:** The initial system message ("You are Gemini, an expert assistant...") can be given once. It will carry across turns. Re-state tone shifts with each prompt if needed (e.g. "Continue as a formal advisor:") but core persona persists.
- **Manage conversation state:** If the chat is very long, occasionally recap relevant points in your prompt ("Earlier, you recommended X; continue building on that.>").
- **Interactive tasks:** For tasks requiring multiple steps (coding, data retrieval, planning), it's often useful to explicitly enumerate steps and wait for each completion. Gemini's "Managed Agents" features allow chaining tools, but even without that, you can issue multi-part prompts and feed back the answer to the next part.
- **Example (turn-by-turn):**
 - **Turn 1 (system):** *"You are a shopping assistant. Help the customer find products."*
 - **Turn 2 (user):** *"I want to buy running shoes."*
 - **Turn 3 (model):** *"Great! What brand or features do you prefer?"* (Gemini, given the system persona, responds appropriately.)
 - **Turn 4 (user):** *"Prefer comfort and support for long runs."*
 - **Turn 5 (model):** *"Based on your needs, I recommend [Product X] because..."* (Uses previous context.)

Tool Use and Function Calling

- **Leverage built-in tools:** Gemini has specialized tools (e.g. a Python code interpreter, web search, calculator). If your prompt needs precise arithmetic or code, enable the **code tool**. E.g. *"Calculate 15% of 237.2 using Python."* Gemini 3 can run Python code to ensure accuracy [17†L850-L858] . For web or data lookup, use the search or file tools if available.
- **Function calling:** The Gemini API supports a function-calling schema (similar to OpenAI's). Provide a JSON schema of the function you want the model to use, and instruct it to output as JSON matching that schema. Examples and official docs (e.g. [Function calling](#)) guide this.
- **Tool writing in output:** If you expect the model to produce code (SQL, JS, HTML, etc.), mention the language and format. E.g. *"Write the SQL query to find X, and output just the query in a code block."* Use explicit tags:

```
# Output
Provide only the SQL query without explanation.
```

- **Chaining tasks:** For multi-step tool use, structure the prompt to "plan" then "execute". E.g.: *"1. Find the prime factors of 345. 2. Use those factors to do Y."* If a step fails (e.g. tool error), the model can often retry differently if prompted to "analyze the error and try again." (See [28] tool use strategy.)

Prompt Templates and Reusable Patterns

- **Master template:** A general effective template is:

```
System Instruction: [Persona + key directives]
Context/Examples: [Data, examples, or conversation history]
Task: [User's specific request or question]
Final Instruction: [Format or tone guidance]
```

This layered approach (role → context → question → format) works for most tasks 【17†L969-L977】 【17†L978-L1007】 . Use bullet lists or numbering in prompts for multiple points.

- **Examples of patterns:**
- **Q&A format:** Useful for extractive tasks. E.g. include “Question: ... Answer:” labels in examples.
- **Table format:** For structured data, show header row in the example and ask to fill rows.
- **JSON schema:** For APIs, provide an example JSON.
- **Persona + task:** Often start with “You are X. [Behavior guidelines]. Then do Y.”
- **Domain-specific prefixes:** E.g. “Act as a medical doctor; analyze the symptoms below and recommend a diagnosis.” Then list symptoms as context.
- **Consistency in style:** If you have a reusable project or series of prompts, keep the language style consistent. Copy the structure once effective, then tweak content.

Debugging and Iteration

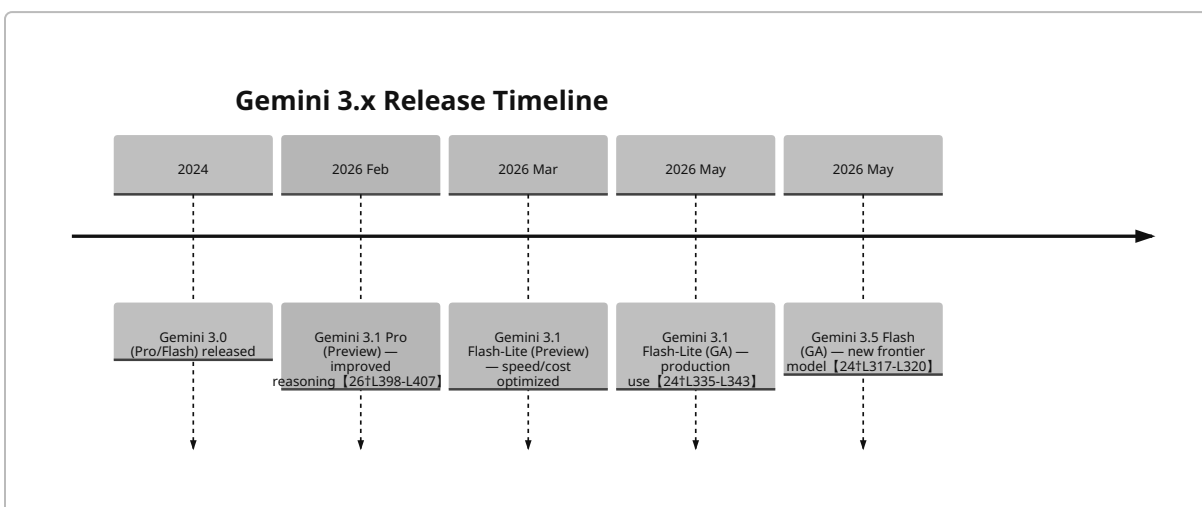
- **Check output vs. examples:** If Gemini’s answer is misformatted or off-target, compare it to your examples. Often it means the example/output pattern wasn’t clear enough. Adjust examples or add another one targeting that case.
- **Incremental testing:** Change one element at a time (e.g. move a constraint from mid-prompt to end) and observe effect. The developer docs suggest the iterative process: try a prompt, then refine 【7†L221-L229】 .
- **Use diagnostic prompts:** Ask the model to **explain its reasoning** or **outline steps** if you suspect misunderstanding. E.g. “Before answering, write out the steps you will take.” Because Gemini can articulate plans if asked explicitly.
- **Temperature experimentation:** Although default is recommended, if responses are too deterministic or stuck, test a higher temperature (e.g. 1.2) to add variability – but do this only after exhausting prompt edits.
- **Monitor token usage:** If Gemini cuts off or omits details, check if you hit token limits. Increase `max_output_tokens` or shorten the question.
- **Use tools for checking:** For factual tasks, use Gemini’s grounding/sourcing mode or supplementary search tools to verify answers.

Version Differences (Gemini 3.0 vs 3.1)

- **Gemini 3.1 Pro improvements:** 3.1 Pro (Feb 2026) brought *better reasoning and complex-task performance* compared to 3.0/3.5 【26†L398-L407】 . All prompt strategies above apply, but 3.1 tends to require less heuristic trickery to get accurate logic.
- **No fundamental syntax change:** Prompt structures that worked in Gemini 3.0 (clear instructions, examples, tags) remain optimal for 3.1. The main difference is performance, not interface.
- **Model aliases:** The latest Gemini 3.x models use names like `gemini-3.1-pro-preview` or `gemini-3.1-flash` (GA). For compatibility, Google often aliases “latest” to the newest version 【25†L515-L524】 . Ensure your API calls target `gemini-3.1-flash` (or Pro/Flash-Lite) to use 3.1.
- **Timeline Highlights:** (A brief timeline of Gemini releases relevant to prompting)

Date	Release/Feature
Nov 2024	Gemini 3.0 (Pro/Flash) debut with advanced reasoning.
Feb 2026	Gemini 3.1 Pro (Preview) – sharper logic on benchmarks 【26†L398-L407】 .
Mar 2026	Gemini 3.1 Flash-Lite (Preview) – optimized for speed/cost.
May 2026	Gemini 3.1 Flash-Lite (GA) – broadly available. Gemini 3.5 Flash (GA) also released for heavy agentic tasks 【24†L335-L343】 【24†L317-L320】 .

(See **Figure: Gemini 3.x Release Timeline** below.)



Best Practices (Summary)

- **Be concise and explicit.** State tasks clearly; avoid fluff 【28†L21-L29】 【19†L58-L67】 . Use bullet lists or numbers for steps.
- **Show desired format.** Use examples to “show, don’t just tell” the output structure 【14†L503-L511】 【19†L73-L82】 .
- **Use tags/headings.** Distinguish sections (role, context, question) with XML tags or Markdown headers 【17†L863-L872】 【28†L79-L88】 .
- **Include context in prompt.** Provide all relevant facts or references *within* the prompt; do not assume knowledge outside it 【30†L248-L257】 【6†L1478-L1487】 .
- **Default parameters.** Keep temperature and sampling at default 1.0/greedy for Gemini 3.x 【6†L1346-L1354】 【35†L13-L22】 .
- **Ground the assistant.** If you need factual answers, say “only use the given info” or similar 【17†L900-L914】 【6†L1453-L1461】 .
- **Few-shot for format.** Always exemplify the answer format with at least one example 【14†L503-L511】 【14†L579-L588】 .

- **Iterate and test.** If results deviate, tweak one element (words, examples, tags) and retest. Keep key instructions at end if they're forgotten 【6†L1414-L1422】 .

Troubleshooting Checklist

- **Wrong format/output:** Ensure examples clearly demonstrate format. Enforce it with an output schema or headings.
- **Too verbose:** Remind the model to “answer concisely” or set persona as terse. Ask for summary.
- **Too short/minimal:** Ask for more detail or specify “explain thoroughly” in the prompt.
- **Hallucinations:** Provide grounding context or split tasks to verify existence before elaboration 【6†L1398-L1407】 .
- **Ignoring constraints:** Check if your “do not do X” was placed early (move it to end) 【6†L1414-L1422】 . Use positive directives instead of negatives 【19†L141-L149】 .
- **Repetition or loops:** Possibly lower the penalty parameters, or shorten response. If it repeats a point, tell it to “move on to the next point.”
- **Cuts off abruptly:** Increase max tokens or break answer into parts.
- **Unhelpful persona:** Reframe the system role to “expert in X,” or remove conflicting persona instructions (Gemini follows its persona closely 【6†L1430-L1438】).
- **Performance drop vs older model:** If behavior changed from Gemini 2.x or Claude, check newer docs. (For example, Gemini 3 dislikes vague prompts that Claude might accept 【31†L99-L107】 .)

References and Resources

Core guidance is drawn from Google’s Gemini developer docs and blog, supplemented by analogous OpenAI/Claude prompt literature. Key sources include Google’s **Gemini API Prompt Design Guide** 【17†L863-L872】 【14†L503-L511】 , the **Gemini 3.1 Pro announcement** 【26†L398-L407】 , and official **prompting best practices** from Google and OpenAI 【6†L1346-L1354】 【19†L139-L148】 . For advanced use, see Google’s docs on [function calling](#), [tool use](#), and [session management](#).
